Recursion

2026-1학기 부경대학교 알고리즘 스터디

목차

- 재귀 (Recursion)
- 백트래킹 (Backtracking)
- 분할 정복 (Divide and Conquer)
- 마스터 정리 (Master Theorem)

재귀 (Recursion)

- 재귀 함수(Recursive)란
  - 함수 내부에서 자기 자신을 다시 호출하는 함수를 의미합니다.
  - 재귀 함수는 수학의 점화식 또는 수학적 귀납법을 코드로 표현한 것 입니다
  - 재귀 함수를 이해하려면 수학적 귀납법의 사고방식을 이해하는 것이 중요합니다

```
void convert_to_binary(int n) {
    if (n == 0) {
        return;
    }

    convert_to_binary(n / 2);
    cout << n % 2;
}
```

귀납법의 구조

- 수학적 귀납법은 모든 자연수(또는 특정 수 이상의 정수)에 대해 어떤 명제가 참임을 증명할 때 사용하는 방법입니다.

1. 가장 작은 자연수에서 명제가 성립함을 확인
   ※ Base Case : 첫 번째 도미노가 넘어간다.
2. n=k 일 때 성립한다고 가정하면, n = k + 1일 때도 성립함을 증명
   ※ Inductive Step : 어떤 도미노가 넘어가면 그 다음 도미노도 반드시 넘어간다.
3. 이 두 조건이 만족되면, 모든 자연수 에 대해 그 명제가 참이라는 결론을 내릴 수 있습니다.
   ※ 결국 모든 도미노가 쓰러진다

재귀의 필수 요소

- 재귀 함수가 무한 루프에 빠지지 않으려면 반드시 두 가지 요소가 필요합니다.
  - 종료 조건 (Base Condition)
  - 더 이상 쪼갤 수 없는 가장 작은 문제에서 재귀를 멈추는 조건
  - 예시 > "카운트다운이 0이 되면 종료한다"

  - 점화식 (Recursive Step)
  - 큰 문제를 더 작은 동일한 문제로 분해하여 자기 자신을 호출
  - 예시 > "N을 출력하고 N-1을 다시 호출한다"

재귀 함수를 짜는 3단계 (N 팩토리얼)

Step 1. 함수의 목적 정의 - 이 함수가 무엇을 계산하는지 명확히 정의

※ N 팩토리얼을 계산하는 함수

Step 2. 종료 조건 설정 - 더 이상 쪼갤 수 없는 가장 작은 문제의 답을 작성

※ N = 1 → 1 반환

Step 3. 점화식 작성 - 함수 f(k)가 올바른 답을 준다고 믿고 f(k+1)을 계산하는 식을 작성

※ 점화식 N! = N × (N-1)!

```
"""
N 팩토리얼(N!)을 계산하여 반환한다.
수학적 점화식: N! = N * (N-1)! (단, 1! = 1)
"""
int factorial(int n) {
    // 1. 종료 조건 (Base Condition): 더 이상 쪼갤 수 없는 가장 작은 문제에서 재귀를 멈춤
    if (n == 1) {
        return 1;
    }

    // 2. 점화식 (Recursive Step): 큰 문제를 더 작은 동일한 문제로 분해하여 자기 자신을 호출
    return n * factorial(n - 1);
}
```

재귀 함수를 대하는 올바른 자세

- X : 함수 호출을 끝까지 추적(Tracing)하려고 하지 말 것
  - 머릿속으로 모든 호출 스택을 그리다 보면 논리가 꼬이고 과부하가 옵니다.
  - 재귀의 깊이가 깊어질수록 인간의 뇌는 이를 완벽히 따라갈 수 없습니다.

- O : 귀납적으로 맞다고 믿고 설계할 것
  - “나보다 작은 문제(n-1)은 이미 누군가(컴퓨터)가 완전하게 풀어왔다” 가정합니다.
  - 나는 그 결과를 이용해 현재의 큰 문제를 어떻게 해결할지만 집중합니다.

PS에서의 재귀 사용 시 주의점

- 메모리 구조 제한 - 재귀 호출시, Stack Frame에 데이터가 쌓입니다.
  - C++ 기준 > 재귀 깊이 100,000 이상 → Stack Overflow 가능
- 함수 호출 자체가 오버헤드를 발생시킵니다.
- 시간 복잡도 문제
  - 특히 중복 호출이 발생하면 시간 복잡도가 폭발합니다.
  - 대표적인 예: > 단순 재귀 피보나치 → O(2^N)

재귀의 장단점

<장점>

- 가독성이 좋다 점화식을 코드에 그대로 옮길 수 있다.
- 상태 관리가 단순하다 복잡한 반복문 변수 관리가 필요 없다.
- 트리 / 그래프 탐색에 유리하고 DFS, 분할정복 구현이 쉽다.

<단점>

- Stack Overflow 위험 - 재귀 깊이가 깊어지면 런타임 에러 발생
- 오버헤드 발생 함수 호출 비용이 반복된다.

반복문 vs 재귀

- 반복문 구현 - 시간복잡도 O(n)

```
int fibo_iterative(int n) {
    if (n <= 1) return n;

    int prev = 0;
    int curr = 1;

    for (int i = 2; i <= n; i++) {
        int next_val = prev + curr;
        prev = curr;
        curr = next_val;
    }

    return curr;
}
```

반복문 vs 재귀

- 재귀 구현 – 시간복잡도 $O(2^n)$
  - 점화식 – F(n) = F(n-1) + F(n-2)

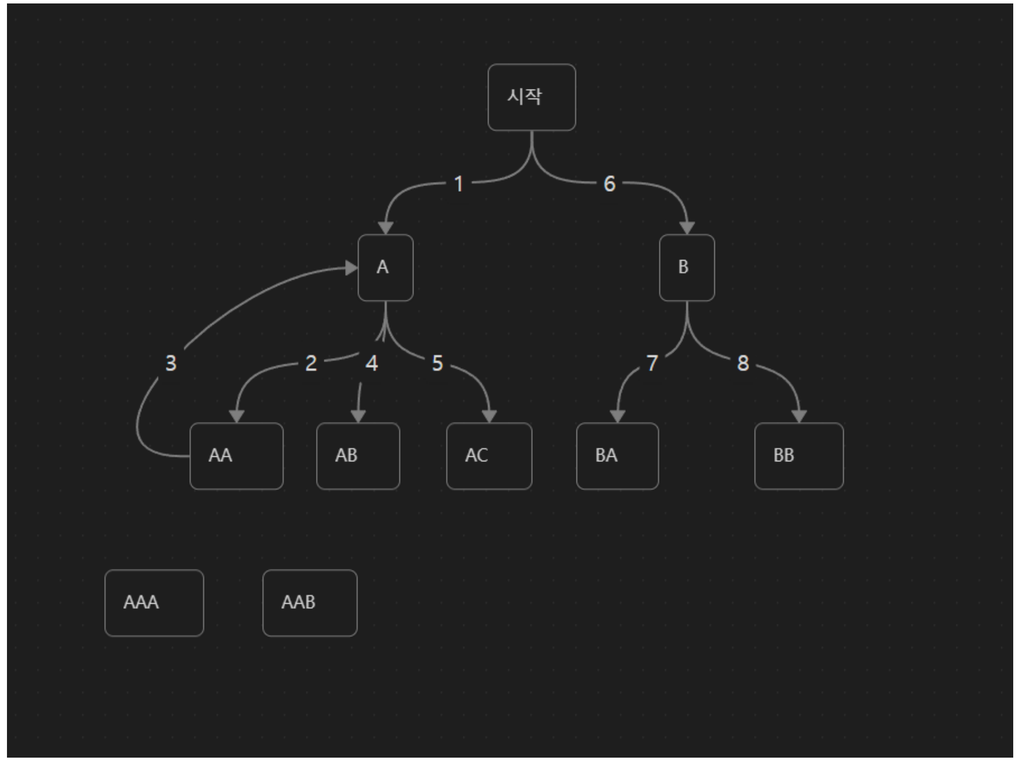
```
int fibo_recursive(int n) {
    if (n <= 1) return n;

    return fibo_recursive(n - 1) + fibo_recursive(n - 2);
}
```

  - 어떻게 하면 시간 복잡도를 낮출 수 있을까?

※ 5장에서 배우는 dp를 적용하면 시간 복잡도를 O(n)까지 낮출 수 있습니다.

백트래킹 (Backtracking)

- 정의
  - 가능한 모든 경우의 수를 탐색하되 조건에 맞지 않는 경로는 미리 포기하는 알고리즘
- 핵심 개념: 가지치기 (Pruning)
  - 탐색 중 가능성이 없는 경로는 더 깊이 탐색하지 않는다.



N과 M (1) BOJ 15649

- 문제
  - 1부터 N까지 자연수 중 중복 없이 M개를 고른 수열 출력하라 ($0 \le M \le N \le 8$)

N과 M (1) BOJ 15649

- 문제
  - 1부터 N까지 자연수 중 중복 없이 M개를 고른 수열 출력하라 ($0 \le M \le N \le 8$)

- 풀이
  - visited[] 배열로 중복을 방지하며, M개를 모두 고르면 결과를 출력합니다.

N과 M (1) BOJ 15649

```
int N, M;
int result[9];
bool visited[9];

// Depth를 기반으로 수열을 탐색하고 출력하는 재귀 함수
void solve(int depth) {
    if (depth == M) {
        for (int i = 0; i < M; i++) {
            std::cout << result[i] << " ";
        }
        std::cout << "\n";
        return;
    }

    for (int i = 1; i <= N; i++) {
        if (visited[i]) {
            continue; // 이미 사용한 숫자는 건너뛰기
        }
        visited[i] = true; // 현재 숫자를 사용 중으로 표시
        result[depth] = i; // 수열에 숫자 저장
        solve(depth + 1); // 다음 자리 숫자를 찾기 위해 재귀 호출
        visited[i] = false; // 탐색 종료 후 다음 경우의 수를 위해 방문 해제
    }
}
```

분할 정복 (Divide and Conquer)

큰 하나의 문제를

1. 나누고 -> 문제를 작은 문제로 분할
2. 각각 해결하고 -> 각 문제를 재귀적으로 해결
3. 결과를 합쳐 해결하는 방법 -> 결과를 합쳐 원래 문제 해결

병합정렬 (Merge Sort)

1. Divide - 배열을 절반으로 분할

2. Conquer - 각 배열을 재귀적으로 정렬

3. Combine - 두 정렬된 배열을 병합

```
void mergeSort(int arr[], int left, int right) {
    // 종료 조건 (Base Condition): 데이터가 1개 이하일 때 재귀 종료
    if (left >= right) {
        return;
    }

    // 분할 (Divide): 중간 지점 계산 (오버플로우 방지 전략 적용)
    int mid = left + (right - left) / 2;

    // 정복 (Conquer): 하위 문제를 해결하기 위한 재귀 호출
    mergeSort(arr, left, mid);      // 왼쪽 절반 정렬
    mergeSort(arr, mid + 1, right);  // 오른쪽 절반 정렬

    // 결합 (Combine): 정렬된 두 배열을 하나로 합침
    merge(arr, left, mid, right);
}
```

병합 정렬 (Merge Sort)

- 배열 [5, 2, 4, 7, 1, 3, 6, 8] 을 병합 정렬한다고 가정해보자.

[5, 2, 4, 7, 1, 3, 6, 8]

[5, 2, 4, 7] [1, 3, 6, 8]

[5, 2] [4, 7] [1, 3] [6, 8]

[5] [2] [4] [7] [1] [3] [6] [8]

분할

Base Condition에 도달하면 합병 시작

[2, 5] [4, 7] [1, 3] [6, 8]

[2, 4, 5, 7] [1, 3, 6, 8]

[1, 2, 3, 4, 5, 6, 7, 8]

합병

병합정렬의 시간복잡도

- 항상 O(Nlog N)보장
  - 데이터를 절반씩 나누는 단계(log N)와 이를 다시 합치는 단계(N)가 결합되어 최상, 평균, 최악의 경우 모두 동일한 성능을 유지합니다.
- 안정적인 성능
  - 퀵 정렬과 달리 입력 데이터의 초기 순서(이미 정렬된 상태 등)에 영향을 받지 않아 어떤 상황에서도 예측 가능한 실행 시간을 보여줍니다.

색종이 만들기 BOJ 2630

- 문제
  - N x N 크기의 종이가 있다. 하얀색(0)과 파란색(1)이 섞여 있는데, 전체가 같은 색이 아니라면 가로세로를 절반으로 잘라 4개의 N/2 x N/2 색종이로 나눈다. 이 과정을 반복해 잘라진 하얀색, 파란색 색종이의 개수를 구하라.

색종이 만들기 BOJ 2630

- 문제
  - N x N 크기의 종이가 있다. 하얀색(0)과 파란색(1)이 섞여 있는데, 전체가 같은 색이 아니라면 가로세로를 절반으로 잘라 4개의 N/2 x N/2 색종이로 나눈다. 이 과정을 반복해 잘라진 하얀색, 파란색 색종이의 개수를 구하라.

- 풀이
  - 현재 탐색 중인 N x N 영역의 색종이가 모두 하얀색(0) 또는 파란색(1)으로 통일되어 있는지 검사합니다.
  - 색이 모두 같다면 더 이상 자를 필요가 없으므로, 해당 색상의 카운트를 1 증가시키고 탐색을 종료합니다.
  - 색이 하나라도 섞여 있다면, 현재 크기 N을 절반(N/2)으로 나눕니다.
  - 잘라진 4개의 사분면(좌상, 우상, 좌하, 우하)을 향해 다시 자기 자신을 호출(재귀)합니다.

색종이 만들기 BOJ 2630

```
int paper[128][128];
int white = 0;
int blue = 0;

"""
주어진 N x N 격자 영역에서 하얀색(0)과 파란색(1) 색종이의 개수를 분할 정복으로 계산한다.
"""
void solve(int y, int x, int size) {
    int color = paper[y][x];
    bool is_same = true;

    // 정복(Conquer) - Base Condition: 현재 영역이 모두 같은 색인지 검사
    for (int i = y; i < y + size; i++) {
        for (int j = x; j < x + size; j++) {
            if (paper[i][j] != color) {
                is_same = false;
                break; // 불필요한 연산 방지 (Early Exit)
            }
        }
        if (!is_same) break;
    }

    if (is_same) {
        // 결합(Combine): 영역 전체가 단일 색상일 경우, 전역 상태에 결과를 누적하여 합산
        if (color == 0) white++;
        else blue++;
    } else {
        // 분할(Divide): 색이 섞여 있다면 크기를 절반(N/2)으로 분할
        int half = size / 2;

        // 정복(Conquer): 분할된 4개의 사분면에 대해 재귀적으로 부분 문제 해결
        solve(y, x, half);
        solve(y, x + half, half);
        solve(y + half, x, half);
        solve(y + half, x + half, half);
    }
}
```

마스터 정리 (Master Theorem)

- 분할 정복 알고리즘의 시간 복잡도를 계산하는 공식

마스터 공식 정의 ($T(N) = aT(N/b) + O(N^d)$)

기호	의미	설명
N	문제 크기	전체 데이터의 입력 규모
a	부분 문제 개수	분할 후 생성되는 하위 작업(재귀 호출)의 수
b	문제 분할 비율	각 단계에서 문제가 줄어드는 비율 (예: 절반으로 나눌 시 2)
d	비용 지수	분할/합치기 비용($O(N^d)$)의 차수 (예: 선형 탐색 시 1)
$O(N^d)$	추가 비용	문제를 나누고 합치는 데 드는 전체 시간 복잡도

복잡도 결정 조건

구분	결정 조건	시간 복잡도	알고리즘 설계적 의미
Case 1	$a > b^d$	$O(N^{\log_b a})$	새끼 문제 생성(a)이 업무 감소(b^d)보다 빠름 → 하위 노드 해결에 시간 집중
Case 2	$a = b^d$	$O(N^d \log N)$	생성 속도와 감소 속도가 일치 (황금 밸런스) → 모든 층에 작업량이 골고루 분배됨
Case 3	$a < b^d$	$O(N^d)$	업무 감소(b^d)가 매우 빨라 하위 작업이 가벼움 → 초기 분할/최종 병합($O(N^d)$)에 시간 집중

마스터 정리 분석

- Case 1: ($a > b^d$)
  - 크기가 N인 문제를 절반 크기(b=2)의 문제 4개(a=4)로 분할하여 해결한다.
  - T(N) = 4T(N/2) + O(1) -> a=4, b=2, d=0
  - $4 > 2^0$ 이므로 Case 1 적용 -> $O(N^2)$

```
void case1_example(int n) {
    if (n <= 1) return;

    // 하위 작업(a)이 4개나 파생되어 재귀 트리의 밑바닥이 무거워짐
    case1_example(n / 2);
    case1_example(n / 2);
    case1_example(n / 2);
    case1_example(n / 2);

    // O(1) 수준의 매우 가벼운 결합 작업
    int x = n * 2;
}
```

마스터 정리 분석

- Case 2: ($a = b^d$)
  - 크기가 N인 문제를 절반(b=2) 크기 2개(a=2)로 분할하고, O(N)의 비용(d=1)으로 결합한다.
  - T(N) = 2T(N/2) + O(N) -> a=2, b=2, d=1,
  - $2 = 2^1$ 이므로 Case 2 적용 -> O(N log N)

```
void case2_example(int n) {
    if (n <= 1) return;

    // 문제를 정확히 2개로 분할하여 재귀 호출
    case2_example(n / 2);
    case2_example(n / 2);

    // O(N)의 비용이 드는 결합 작업 (각 층마다 동일한 무게의 작업량 발생)
    for (int i = 0; i < n; i++) {
        // 결합 로직 수행
    }
}
```

마스터 정리 분석

- Case 3: ($a < b^d$)
  - 크기가 N인 문제를 절반(b=2) 크기 2개(a=2)로 분할하지만, 병합에 $O(N^2)$의 큰 비용(d=2)이 든다.
  - $T(N) = 2T(N/2) + O(N^2)$ -> a=2, b=2, d=2
  - $2 < 2^2$ 이므로 Case 3 적용 -> $O(N^2)$

```
void case3_example(int n) {
    if (n <= 1) return;

    // 파생되는 하위 문제는 2개뿐이라 상대적으로 빠르게 처리됨
    case3_example(n / 2);
    case3_example(n / 2);

    // O(N^2)의 매우 무거운 결합 작업 (이 부분의 비용이 전체 시간을 지배함)
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            // 복잡한 결합 로직 수행
        }
    }
}
```

그 외 연습 문제

- 15650 N과 M (2)
- 9663 N-Queen
- 1074 Z
- 1629 곱셈